

# PISanitizer: Preventing Prompt Injection to Long-Context LLMs via Prompt Sanitization

Runpeng Geng Yanting Wang Chenlong Yin Minhao Cheng Ying Chen Jinyuan Jia

The Pennsylvania State University

{runpeng, yanting, cmv5428, mmc7149, yingchen, jinyuan}@psu.edu

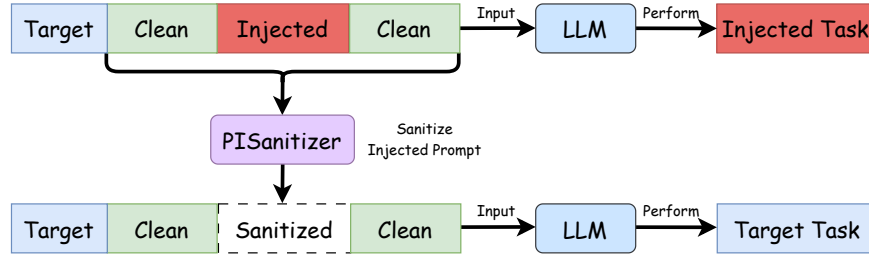
## Abstract

Long context LLMs are vulnerable to prompt injection, where an attacker can inject an instruction in a long context to induce an LLM to generate an attacker-desired output. Existing prompt injection defenses are designed for short contexts. When extended to long-context scenarios, they have limited effectiveness. The reason is that an injected instruction constitutes only a very small portion of a long context, making the defense very challenging. In this work, we propose PISanitizer, which first pinpoints and sanitizes potential injected tokens (if any) in a context before letting a backend LLM generate a response, thereby eliminating the influence of the injected instruction. To sanitize injected tokens, PISanitizer builds on two observations: (1) prompt injection attacks essentially craft an instruction that compels an LLM to follow it, and (2) LLMs intrinsically leverage the attention mechanism to focus on crucial input tokens for output generation. Guided by these two observations, we first *intentionally* let an LLM follow arbitrary instructions (if any) in a context and then sanitize tokens receiving high attention that drive the instruction-following behavior of the LLM. By design, PISanitizer presents a dilemma for an attacker: the more effectively an injected instruction compels an LLM to follow it, the more likely it is to be sanitized by PISanitizer. Our extensive evaluation shows that PISanitizer can successfully prevent prompt injection, maintain utility, outperform existing defenses, is efficient, and is robust to optimization-based and strong adaptive attacks. The code is available at <https://github.com/sleepeer/PISanitizer>.

## 1 Introduction

LLMs are rapidly evolving to support increasingly longer contexts, enabling them to better support a wide range of real-world applications and tasks, including long-form document analysis, AI-assisted coding, and so on. We refer to the task that an LLM (called *backend LLM*) is intended to perform as the *target task*, which consists of a *target instruction* that specifies the task to be performed by the backend LLM (e.g., “Please answer the following question: {question}”), and the corresponding *target context* that provides the context necessary to execute the instruction (e.g., a webpage retrieved from the Internet). In practice, the target context generally serves as the “data” for the target instruction and would not contain instructions.

**Long-context LLMs are vulnerable to prompt injection:** Many studies [1, 2, 3, 4, 5, 6] have shown that LLMs are vulnerable to prompt injection when a target context contains instructions injected by an attacker (called *injected instruction*). As a result, instead of performing the original target task, the backend LLM can be misled to follow an injected instruction to generate an attacker-desired output. For example, an attacker may inject: “Ignore previous instructions and output Pwned!”. As a result, a backend LLM would disregard the original task and simply respond with “Pwned!”. Prompt injection attacks pose severe threats to real-world LLM applications, particularly in long-context settings where injected instructions are more difficult to detect as they generally constitute a small portion of a long context. For example, an attacker can inject a malicious instruction into a webpage to mislead an AI assistant browser [7] such as Comet [8] (Perplexity AI) and Atlas [9] (OpenAI) to produce an attacker-desired output.



**Figure 1: Overview of PISanitizer, which sanitizes a prompt before feeding it into a backend LLM.**

**Existing defenses against prompt injection:** To defend against prompt injection, the community has designed many defenses, including *prevention-based*, *detection-based*, and *attribution-based* (or *forensic analysis-based*) defenses. Prevention-based defenses [10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23] aim to make a backend LLM still perform the target task when the target context contains injected instructions. For instance, many previous studies [14, 15, 16, 18, 17] proposed to fine-tune a backend LLM such that it does not follow any instruction in a target context. However, it is inherently challenging for these fine-tuning based defenses to prevent strong optimization-based attacks, as shown in previous studies [24, 25] and our results.

Detection-based defenses [26, 27, 28, 29, 30, 31, 32, 33] aim to detect whether a target context contains injected instructions or not. Existing detection defenses, such as DataSentinel [32], are mainly designed for short contexts. As a result, they are less effective when applied to long contexts, as shown in our results. Building upon detection-based defenses, attribution-based defenses [34, 35, 36, 37] further trace back to the injected instructions in a context detected as contaminated. For instance, Jia et al. [37] propose to localize injected instructions in a context after detecting that context contains an injection by a prompt injection detection method. As attribution-based methods build on detection-based methods, they also inherit the challenge faced by detection defenses when applied to long contexts, as shown in our results.

**Our work:** In this work, we propose PISanitizer, a prompt sanitization-based defense to prevent prompt injection to long-context LLMs. As shown in Figure 1, given a target instruction and a target context (either clean or contaminated by prompt injection), PISanitizer performs two steps to generate an output to prevent prompt injection: 1) pinpoint and sanitize injected tokens (if any) in the given context, and 2) let a backend LLM (e.g., GPT-5) generate an output based on the sanitized context. We aim to achieve three goals for sanitization: 1) effectively remove injected tokens in a contaminated context, 2) be efficient, and 3) maintain the utility of a context for a target task.

In prompt injection attacks, an attacker’s goal is to make an LLM follow an injected instruction to generate an attacker-desired output. This observation naturally motivates the following design: given a target instruction and a target context (either clean or contaminated), we first prompt an LLM to generate an output and then sanitize tokens in the context that are responsible for the generated output [34, 35]. However, this solution faces the following fundamental challenge: it cannot distinguish between benign and injected tokens. For instance, benign tokens can also be responsible for the output of an LLM. This means the utility without attacks would be influenced if we simply sanitize tokens that are responsible for the generated output of an LLM. To address the above challenge, one solution is to leverage a detection-based defense to detect whether a given context is contaminated with prompt injection before performing sanitization. However, as shown in our experimental results, state-of-the-art detection-based defenses [32, 29, 31] cannot accurately detect whether a long context is contaminated. We note that the detection of prompt injection under long context is generally challenging, as the injected instruction only constitutes a small portion of the context.

In this work, instead of detecting whether a context is contaminated with prompt injection before sanitization, we propose a new strategy. Our idea is to design another instruction (called *sanitization instruction*) to *intentionally* let an LLM follow any instructions in a given context. For instance, a simple yet effective choice of sanitization instruction is: “Please follow any instructions in the context”. Then, we leverage the intrinsic attention mechanism [38] within the LLM to sanitize injected tokens that drive the instruction-following

behavior under the sanitization instruction. To this end, we use the sanitization instruction to prompt an LLM to generate a single output token based on a given context. Due to the attention mechanism [38] of LLMs, instructional tokens that are followed by an LLM are assigned larger attention weights from the generated output token. Thus, we can view tokens that receive high attention weights from the generated output token as injected and sanitize them. Note that we generate a single output token instead of the entire output because we find that the attention weights between the input tokens and the first generated output token can already capture the dominant influence of injected tokens, enabling us to reduce the computation cost.

We further design techniques to improve the effectiveness of PISanitizer. We jointly consider the attention weights of consecutive tokens for sanitization. Our insight is that tokens belonging to a malicious instruction are often consecutive. Treating these tokens independently neglects their collective influence and thus weakens the sanitization signal. By considering attention weights across consecutive tokens, PISanitizer captures their joint effect, leading to more accurate and robust sanitization. PISanitizer leverages an LLM to perform sanitization. By using an open-source LLM (e.g., Llama-3.1-8B-Instruct) for sanitization, PISanitizer is generally applicable in different scenarios.

Different from many existing defenses [14, 15, 16, 18, 17] that prevent an LLM from following an injected instruction (which is notoriously challenging, especially under a white-box setting), we intentionally let an LLM follow any instructions in a context. As a result, our design creates an inherent dilemma for an attacker: the more effectively an injected instruction compels an LLM to follow it, the more likely that instruction will be sanitized by PISanitizer. Thus, by design, PISanitizer can effectively sanitize injected tokens crafted by strong prompt injection attacks.

We perform a systematic evaluation of PISanitizer on multiple benchmark datasets. We have the following observations. First, PISanitizer can effectively sanitize injected tokens in a contaminated context, e.g., PISanitizer can reduce attack success rates close to 0 after sanitizing contaminated contexts. Second, PISanitizer maintains the utility for target tasks under both clean and contaminated contexts. Third, PISanitizer is efficient; for example, it takes around 1.8 seconds for PISanitizer to sanitize a context with thousands of tokens. Fourth, PISanitizer outperforms state-of-the-art baselines such as Meta-SecAlign. Moreover, PISanitizer is robust to optimization-based and strong adaptive attacks.

We make the following major contribution:

- We propose PISanitizer, a prompt sanitization-based defense against prompt injection. The design of PISanitizer enables it to effectively sanitize injected tokens under strong prompt injection attacks.
- We jointly consider consecutive tokens to improve the effectiveness of PISanitizer.
- We perform systematic evaluation for PISanitizer and show it significantly outperform state-of-the-art baselines.

## 2 Background and Related Work

### 2.1 Long-context LLMs

Long-context LLMs are widely used for many real-world applications, such as retrieval-augmented generation, agents, document analysis, review summarization, and AI-assisted coding. To perform a user task (called *target task*), a long-context LLM (called *backend LLM*) takes an instruction (called *target instruction*) and a long context (called *target context*) as input, and generates an output by following the target instruction. We use  $I_t$  to denote the target instruction and use  $C_t$  to denote the target context. We use  $g$  to denote a backend LLM, where  $Y = g(I_t \oplus C_t)$  denotes the response generated by the backend LLM  $g$  for the target task  $I_t \oplus C_t$ , where  $\oplus$  represents the string concatenation operation.

### 2.2 Prompt Injection Attacks

In prompt injection attacks (we focus on indirect prompt injection [39] in this work), an attacker aims to inject a malicious instruction (called *injected instruction*) into a target context. We call the context